Ruby for MSE 2 (Unit 3 & Unit 4)

Unit -3 [Rails Models & User Management]

1. ActiveRecord ORM & Migrations

What is ActiveRecord?

ActiveRecord is the **M (Model) in MVC**. It is an **Object-Relational Mapping (ORM)** system that acts as a **bridge between Ruby code and the database**.

Instead of writing raw SQL:

```
SELECT * FROM users WHERE id = 1;
```

You write Ruby:

```
User.find(1)
```

ORM Mapping:

Database	Rails
Table (users)	Ruby Class (User)
Row	Ruby Object
Column	Attribute

What are Migrations?

Migrations are a **DSL for managing database schema over time** using Ruby instead of SQL. Every structural change (create table, add column, add index) is written as a migration file stored in db/migrate/.

Example

rails generate migration CreatePosts title:string body:text user:references

```
class CreatePosts < ActiveRecord::Migration[7.0]
  def change
    create_table :posts do |t|
```

```
    t.string :title
    t.text :body
    t.references :user, null: false, foreign_key: true
    t.timestamps  # adds created_at & updated_at
   end
 end
end
```

Run with:

rails db:migrate

Important Migration Commands

Command	Purpose
rails db:migrate	Run pending migrations
rails db:rollback	Undo last migration
rails db:migrate:status	Check migration status

Key Takeaway

ActiveRecord lets you **interact with the database using Ruby**, and Migrations let you **version control your database schema** — both together make database management clean, readable, and collaborative without writing raw SQL.

2. Validations and Callbacks

Validations

Validations ensure that **only valid data is saved to your database**. They act as a safety check before any record is stored.

Why Validations?

- Prevent bad/incomplete data from entering the database
- Show proper error messages to users
- Runs **before** saving to the database

Example:

class Post < ApplicationRecord

```
  validates :title, presence: true, length: { minimum: 5 }
  validates :body, presence: true
end
```

Common Validation Helpers:

Validation	Purpose
presence: true	Field cannot be blank
length: { minimum: 5 }	Minimum character length
uniqueness: true	No duplicate values
numericality: true	Must be a number
format: { with: /regex/ }	Must match a pattern

Callbacks

Callbacks are **methods that get called at certain moments of an object's life cycle** — like before saving, after creating, before destroying, etc.

Why Callbacks?

- Automate tasks during object lifecycle
- Keep model logic clean and centralized
- No need to manually call these methods every time

Example:

```
class User < ApplicationRecord
  before_save :normalize_email

  private

  def normalize_email
    self.email = email.downcase.strip
  end
end
```

Here, every time a User is saved, the email is **automatically converted to lowercase and stripped of spaces**.

Common Callbacks:

Callback	When it Runs
before_save	Just before saving (create or update)
after_save	Just after saving
before_create	Just before first save (create only)
after_create	Just after record is created
before_destroy	Just before record is deleted
after_destroy	Just after record is deleted

Key Difference

	Validations	Callbacks
Purpose	Check data correctness	Automate lifecycle actions
When	Before save/create	At various lifecycle moments
Example	Title must be present	Downcase email before save

Key Takeaway

- **Validations** protect your database from invalid data.
- **Callbacks** automate actions at key points in an object's lifecycle.
- Both are defined inside the **Model** and keep your application logic clean and organized.

3. Model Relationships (Associations)

What are Associations?

Associations allow you to **define connections between different models**. Instead of writing complex SQL joins, Rails handles it automatically through simple declarations in the model.

Types of Associations

1. One-to-Many (has_many / belongs_to)

One User can have **many Posts**, but each Post belongs to **one User**.

```
# app/models/user.rb
class User < ApplicationRecord
  has_many :posts, dependent: :destroy
end

# app/models/post.rb
class Post < ApplicationRecord
  belongs_to :user
end
```

- dependent: :destroy — if a User is deleted, all their posts are also deleted automatically.

2. Many-to-Many (has_many :through)

One Post can have **many Tags**, and one Tag can belong to **many Posts**.

```
# Join table model
class PostTag < ApplicationRecord
  belongs_to :post
  belongs_to :tag
end

class Post < ApplicationRecord
  has_many :post_tags
  has_many :tags, through: :post_tags
end

class Tag < ApplicationRecord
  has_many :post_tags
  has_many :posts, through: :post_tags
End
```

3. One-to-One (has_one / belongs_to)

One User has **exactly one** Profile.

```
class User < ApplicationRecord
```

```
  has_one :profile
end

class Profile < ApplicationRecord
  belongs_to :user
end
```

Association Methods Rails Provides

Once associations are defined, Rails gives you **helper methods automatically**:

Method	What it Does
user.posts	Get all posts of a user
user.posts.create(...)	Create a post for that user
post.user	Get the user of a post
user.posts.destroy_all	Delete all posts of a user

Summary Table

Association	Meaning	Example
has_many	One to Many	User has many Posts
belongs_to	Opposite of has_many	Post belongs to User
has_one	One to One	User has one Profile
has_many :through	Many to Many via join table	Post has many Tags

Key Takeaway

Rails follows **"Convention over Configuration"** — by simply declaring has_many :posts, Rails automatically knows how to write the SQL joins and handle foreign keys. Associations make your code **clean, readable, and powerful** without writing complex queries manually.

4. User Management with Devise

What is Devise?

Devise is the **industry standard authentication gem for Rails**. It provides a complete user management system out of the box including:

- User Sign-up & Login
- Password Reset
- Session Management
- Access Restriction

Setup Steps

Step 1 — Add Devise to Gemfile

gem 'devise'

Then run:

bundle install

Step 2 — Install Devise

rails generate devise:install

Step 3 — Generate User Model

rails generate devise User

Step 4 — Run Migration

rails db:migrate

What Devise Generates?

Once set up, Devise automatically creates:

Feature	What it Provides
Routes	/users/sign_in, /users/sign_up, /users/sign_out

Views	Login form, Sign-up form, Password reset form
Model	User model with encrypted password
Helpers	current_user, user_signed_in?

Restricting Access (Authentication)

You can restrict actions so only **logged-in users** can perform them:

```
class PostsController < ApplicationController
  before_action :authenticate_user!, except: [:index, :show]

  def create
    @post = current_user.posts.build(post_params)
    if @post.save
      redirect_to @post
    end
  end
end
```

- authenticate_user! — redirects to login page if not signed in
- except: [:index, :show] — allows everyone to view posts
- current_user — gives the currently logged in user object

Sign-in / Sign-up Links in Views

```
# app/views/layouts/application.html.erb
<% if user_signed_in? %>
  Logged in as: <%= current_user.email %>
  <%= link_to 'Logout', destroy_user_session_path,
      data: { "turbo-method": :delete } %>
<% else %>
  <%= link_to 'Login', new_user_session_path %>
  <%= link_to 'Sign Up', new_user_registration_path %>
<% end %>
```

Important Devise Helpers

Helper	Purpose
current_user	Returns the logged-in user object
user_signed_in?	Returns true if user is logged in
authenticate_user!	Forces login before accessing action
destroy_user_session_path	Route for logout
new_user_session_path	Route for login page
new_user_registration_path	Route for sign-up page

Key Takeaway

Devise removes the complexity of building authentication from scratch. With just a few commands, you get a **complete, secure user management system** — sign-up, login, logout, and access control — all ready to use in your Rails application.

5. Implementation: Simple Blog Example

Overview

This example combines **ActiveRecord, Associations, Validations, and Devise** to build a simple blog where **users own posts**.

Step 1 — Models & Associations

```
# User Model
class User < ApplicationRecord
  has_many :posts, dependent: :destroy
end

# Post Model
class Post < ApplicationRecord
  belongs_to :user
  validates :title, presence: true, length: { minimum: 5 }
  validates :body, presence: true
end
```

Step 2 — The Form with Validations

```
# app/views/posts/_form.html.erb
<%= form_with(model: post) do |f| %>

  <% if post.errors.any? %>
    <%= pluralize(post.errors.count, "error") %>
    prohibited this post from being saved:
    <% post.errors.full_messages.each do |message| %>
      <%= message %>
    <% end %>
  <% end %>

  <%= f.label :title %>
  <%= f.text_field :title %>

  <%= f.label :body %>
  <%= f.text_area :body %>

  <%= f.submit %>
<% end %>
```

- If validations fail, **error messages are shown** to the user
- post.errors.full_messages — lists all validation errors

Step 3 — Controller with Devise Protection

```
class PostsController < ApplicationController
  before_action :authenticate_user!, except: [:index, :show]

  def create
    @post = current_user.posts.build(post_params)
    if @post.save
      redirect_to @post
    end
  end

  private

  def post_params
    params.require(:post).permit(:title, :body)
  end
end
```

- authenticate_user! — only logged in users can create posts
- current_user.posts.build — post is automatically linked to logged in user

Step 4 — Sign-up / Login Links in Layout

```
# app/views/layouts/application.html.erb
<% if user_signed_in? %>
  Logged in as: <%= current_user.email %>
  <%= link_to 'Logout', destroy_user_session_path,
     data: { "turbo-method": :delete } %>
<% else %>
  <%= link_to 'Login', new_user_session_path %>
  <%= link_to 'Sign Up', new_user_registration_path %>
<% end %>
```

Complete Flow

User Signs Up / Logs In (Devise)
↓
User Fills Post Form (Views)
↓
Validations Check (Model)
↓
Post Saved with user_id (ActiveRecord)
↓
Post Listed on Blog (Controller)

Key Takeaway

This simple blog example demonstrates how all Rails concepts work **together in one application**:

- **Devise** handles authentication
- **Associations** link posts to users
- **Validations** ensure clean data
- **ActiveRecord** manages database operations
- **Convention over Configuration** keeps everything clean and minimal

Unit 4 [PostgreSQL & Ruby on Rails]

1. PostgreSQL Architecture

What is PostgreSQL?

PostgreSQL is a **powerful, open-source relational database system**. It uses a **Client-Server model** where the client sends requests and the server processes them.

Client-Server Model

Client (psql / pgAdmin / Rails App)
↓
Postmaster (Supervisor Process)
↓
Backend Process (one per connection)
↓
Database

- **Postmaster** — The main supervisor process that manages the entire database cluster
- **Process-per-connection** — For every client connection, a **new backend process is forked**. This ensures **isolation and stability**

Memory Structure

Component	Purpose
Shared Buffers	Caches frequently used data pages in memory
WAL Buffers	Temporarily stores transaction log entries before writing to disk

MVCC (Multi-Version Concurrency Control)

MVCC allows **readers and writers to operate simultaneously without locking** each other.

- Each transaction sees a **"snapshot"** of the data at that point in time
- **Readers don't block Writers** and **Writers don't block Readers**
- This makes PostgreSQL highly efficient for **concurrent applications**

Transaction 1 (Reading) → Sees snapshot at T1
Transaction 2 (Writing) → Creates new version
↓
No Locks — Both run simultaneously

Write-Ahead Logging (WAL)

WAL ensures **data is never lost** even if the system crashes.

- All changes are **written to logs BEFORE actual data files are updated**
- This ensures **Durability** — the **D in ACID**
- If a crash happens, PostgreSQL can **recover using WAL logs**

Change Request
↓
Write to WAL Log first
↓
Then Write to Data File
↓
Data is Safe & Durable

ACID Properties

PostgreSQL fully supports ACID:

Property	Meaning
A — Atomicity	All operations succeed or none do
C — Consistency	Data always remains in a valid state
I — Isolation	Transactions don't interfere with each other
D — Durability	Committed data is never lost (ensured by WAL)

Summary of Architecture

Component	Role
Postmaster	Supervisor that manages the DB cluster
Backend Process	Handles each client connection separately

Shared Buffers	In-memory cache for data pages
WAL Buffers	In-memory buffer for transaction logs
MVCC	Allows concurrent read/write without locks
WAL	Ensures durability and crash recovery

Key Takeaway

PostgreSQL's architecture is designed for **reliability, concurrency, and performance**. The combination of **MVCC for concurrency** and **WAL for durability** makes it one of the most trusted databases for production applications like Rails.

2. Data Types in PostgreSQL

What are Data Types?

Data Types define **what kind of data a column can store** in a PostgreSQL table. Choosing the right data type ensures **better performance, storage efficiency, and data integrity**.

1. Numeric Types

Type	Description
INT	Whole numbers (e.g., age, count)
BIGINT	Large whole numbers
DECIMAL	Exact decimal numbers (e.g., price)
SERIAL	Auto-incrementing integers — used for **primary keys / IDs**

```
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  age INT,
  balance DECIMAL
);
```

2. String Types

Type	Description
VARCHAR(n)	Variable length string with limit
TEXT	Unlimited length string — **preferred in PostgreSQL**

```
CREATE TABLE posts (
  title VARCHAR(100),
  body TEXT
);
```

> **Note:** TEXT is unlimited and performant in PostgreSQL — preferred over VARCHAR for long content.

3. Boolean Type

Type	Description
BOOLEAN	Stores TRUE, FALSE, or NULL

```
CREATE TABLE users (
  active BOOLEAN DEFAULT TRUE
);
```

4. Temporal (Date & Time) Types

Type	Description
DATE	Stores only date (year, month, day)
TIMESTAMP	Stores date and time (no timezone)
TIMESTAMPTZ	Stores date and time **with timezone**

```
CREATE TABLE events (
  event_date DATE,
  created_at TIMESTAMPTZ
);
```

> **Best Practice:** Always prefer TIMESTAMPTZ for **global applications** to avoid timezone issues.

5. Advanced Types

Type	Description
JSONB	Binary JSON — **indexed and faster** than plain JSON
UUID	Universally Unique Identifier — used for secure IDs
ARRAY	Stores multiple values in a single column

```
CREATE TABLE profiles (
  id UUID PRIMARY KEY,
  settings JSONB,
  tags TEXT[]
);
```

> **Note:** JSONB is indexed binary JSON — much faster for querying than plain JSON.

Summary Table

Category	Types	Key Point
Numeric	INT, BIGINT, DECIMAL, SERIAL	SERIAL for auto-increment IDs
String	VARCHAR(n), TEXT	TEXT is preferred in PostgreSQL
Boolean	BOOLEAN	TRUE, FALSE, or NULL
Temporal	DATE, TIMESTAMP, TIMESTAMPTZ	Use TIMESTAMPTZ for global apps
Advanced	JSONB, UUID, ARRAY	JSONB is indexed binary JSON

Key Takeaway

Choosing the **right data type** is important for performance and data integrity. PostgreSQL offers powerful advanced types like **JSONB and UUID** that go beyond standard SQL — making it ideal for modern web applications like Rails.

3. SQL Basics (CRUD)

What is CRUD?

CRUD stands for **Create, Read, Update, Delete** — the four basic operations performed on a database. Every web application essentially performs these four operations on data.

1. CREATE — INSERT

Used to **add new records** into a table.

```
INSERT INTO users (name, email)
VALUES ('John Doe', 'john@example.com')
RETURNING id;
```

Keyword	Purpose
INSERT INTO	Specifies the table to insert into
VALUES	The actual data to insert
RETURNING id	Returns the newly created record's ID

2. READ — SELECT

Used to **fetch/retrieve records** from a table.

```
-- Get all active users
SELECT * FROM users
WHERE active = true
ORDER BY created_at DESC;
```

Common SELECT Clauses:

Clause	Purpose
SELECT *	Fetch all columns
SELECT name, email	Fetch specific columns
WHERE	Filter records by condition

ORDER BY	Sort results (ASC / DESC)
LIMIT	Restrict number of results

```
-- More examples
SELECT name, email FROM users;
SELECT * FROM users LIMIT 10;
SELECT * FROM users ORDER BY name ASC;
```

3. UPDATE

Used to **modify existing records** in a table.

```
UPDATE users
SET last_login = NOW()
WHERE id = 1;
```

Keyword	Purpose
UPDATE	Specifies the table to update
SET	The column and new value to update
WHERE	Condition to find the specific record
NOW()	Built-in function for current timestamp

> **Important:** Always use WHERE with UPDATE — without it, **all records will be updated**.

4. DELETE

Used to **remove records** from a table.

```
DELETE FROM users
WHERE email LIKE '%@spam.com';
```

Keyword	Purpose
DELETE FROM	Specifies the table to delete from
WHERE	Condition to find records to delete
LIKE	Pattern matching (% is wildcard)

> **Important:** Always use WHERE with DELETE — without it, **all records will be deleted**.

LIKE Pattern Matching

Pattern	Matches
LIKE '%@spam.com'	Anything ending with @spam.com
LIKE 'John%'	Anything starting with John
LIKE '%doe%'	Anything containing doe

Complete CRUD Example

```
-- CREATE
INSERT INTO users (name, email)
VALUES ('John Doe', 'john@example.com')
RETURNING id;

-- READ
SELECT * FROM users
WHERE active = true
ORDER BY created_at DESC;

-- UPDATE
UPDATE users
SET last_login = NOW()
WHERE id = 1;

-- DELETE
DELETE FROM users
WHERE email LIKE '%@spam.com';
```

Key Takeaway

CRUD operations are the **foundation of every database-driven application**. In Rails, ActiveRecord performs these same operations behind the scenes using Ruby code — but understanding the raw SQL helps you write **better, optimized queries** for your application.

4. Joins: Deep Dive

What are Joins?

Joins allow you to **retrieve related data from multiple tables** in a single query. Instead of making separate queries for each table, joins combine them efficiently.

Why Joins?

Consider two tables:

users table

id	name
1	John
2	Jane

posts table

id	title	user_id
1	Post A	1
2	Post B	1
3	Post C	NULL

1. INNER JOIN

Returns rows **only when there is a match in BOTH tables**. Non-matching records are excluded.

```
SELECT users.name, posts.title
FROM users
INNER JOIN posts ON users.id = posts.user_id;
```

Result:

name	title
John	Post A
John	Post B

> Jane is excluded because she has no posts. Post C is excluded because it has no user.

2. LEFT JOIN (Outer Join)

Returns **all records from the LEFT table**, and matched records from the right table. Unmatched right records result in **NULL**.

```
SELECT users.name, posts.title
FROM users
LEFT JOIN posts ON users.id = posts.user_id;
```

Result:

name	title
John	Post A
John	Post B
Jane	NULL

> Jane is included even though she has no posts — her post title shows as NULL.

3. FULL OUTER JOIN

Returns **all records when there is a match in EITHER table**. Non-matching records show NULL on the missing side.

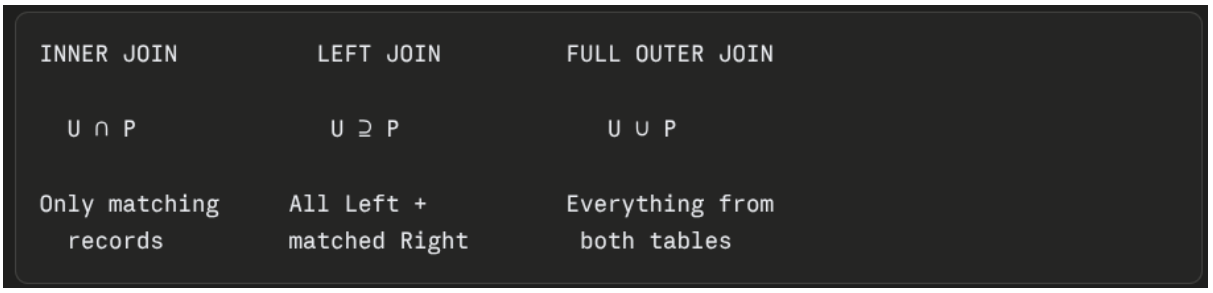
```
SELECT * FROM users
FULL OUTER JOIN posts ON users.id = posts.user_id;
```

Result:

name	title
John	Post A
John	Post B
Jane	NULL
NULL	Post C

> Both Jane (no posts) and Post C (no user) are included with NULL on missing side.

Visual Comparison



Joins in Rails (ActiveRecord)

Rails handles joins automatically through associations:

```
# INNER JOIN in Rails
@published_authors = User.joins(:posts)
                         .where(posts: { status: 'published' })

# LEFT JOIN in Rails
@users_with_posts = User.left_joins(:posts)
                        .select('users.*, posts.title')
```

Summary Table

Join Type	Returns	NULL for
INNER JOIN	Only matching rows from both tables	Excludes non-matching
LEFT JOIN	All left rows + matched right rows	Unmatched right side
FULL OUTER JOIN	All rows from both tables	Both unmatched sides

Key Takeaway

Joins are essential for working with **relational databases**. Use **INNER JOIN** when you need only matching data, **LEFT JOIN** when you need all records from one table, and **FULL OUTER JOIN** when you need everything from both tables. In Rails, ActiveRecord handles joins automatically through **associations and the joins() method**.